PALMETTO

A burst-hardened, one-way FSK messaging protocol for VHF FM — N4OMG

PALMETTO takes its name from Fort Moultrie, built in 1776 on Sullivan's Island at the mouth of Charleston Harbor. The fort's walls were made of palmetto logs, and when the Royal Navy bombarded it, the spongy palmetto wood absorbed the cannonballs instead of shattering — the fort held, and the palmetto went on the state flag. This protocol is built on the same idea. It does not ask for retransmission when the channel damages a frame; it is engineered to absorb the damage. A full-frame interleaver spreads burst errors thin, a soft-decision Viterbi decoder corrects them, and when interference destroys an entire copy of a message, chase combining recovers the payload from the accumulated evidence of every copy received. Bursts hit the frame; nothing shatters.

1. Summary

PALMETTO is a one-way, store-on-arrival text messaging and telemetry protocol for narrowband VHF FM voice channels, using audio frequency-shift keying (AFSK) on the standard Bell 202 tone pair. The architecture is deliberately simplex: there is no return channel, no ACK, and no handshake. The transmitter sends a message one or more times and stops. The receiver runs unattended around the clock, like a pager or a mailbox — it detects an incoming frame on its own, captures it, decodes it, and holds the message until the operator reads it.

Reliability comes from forward error correction and time diversity rather than retransmission requests. Each frame carries a NASA-standard K=7 rate-1/2 convolutional code decoded with a soft-decision Viterbi algorithm, spread by a full-frame square block interleaver. When the transmitter repeats a frame, the receiver first tries each copy on its own; if every copy fails, it sums their soft-decision streams (maximal-ratio chase combining) and decodes the sum — so two copies that are each individually unreadable can still yield a perfect decode. This is the benefit of hybrid-ARQ redundancy without needing a return path.

Because the link is one-way and the receiver is always listening, PALMETTO suits any application where the far end cannot, or should not, have to talk back: environmental sensor and telemetry beacons, ship-to-ship and ship-to-shore messaging, remote weather stations, and — with suitable equipment — ground-to-satellite uplinks.

2. Details

Physical layer

Binary AFSK at 1200 Hz (mark) and 2400 Hz (space) — the Bell 202 tone pair — generated as 48 kHz PCM audio and passed through an ordinary FM voice transceiver. Baud rate is a compile-time parameter; 100 and 300 baud are the proven variants (PALMETTO-100 and PALMETTO-300), with all correlator and timing constants derived from the symbol length so a baud change is a one-line edit on both ends. The modulator keeps a continuous phase accumulator and applies raised-edge shaping at every symbol boundary to keep the occupied bandwidth clean.

Frame format

```
[ 128-bit preamble ] [ 16-bit sync word 0xD3D3 ] [ 60-bit coded length header ]
[ R × C interleaved coded body ]
```

The preamble is a raw alternating bit pattern the receiver uses to settle its symbol clock; the sync word anchors byte alignment. The body carries the message text packed 7 bits per character (a free 12.5% airtime saving for ASCII) followed by a CRC-16 over the original text, which is the final arbiter of a good decode.

Forward error correction

The body and header are encoded with the K=7, rate-1/2 convolutional code using the NASA-standard generator pair 171/133 (octal) — the same code flown on Voyager — with free distance 10 and the trellis flushed to the zero state so the decoder can anchor its traceback. The receiver runs a full soft-decision Viterbi decoder: each demodulated bit carries a bipolar confidence value, low-reliability bits are floored to true erasures, and the branch metric is a linear correlation, worth roughly 5 dB of coding gain versus about 1.5 dB for the hard-decision Hamming(7,4) scheme it replaced.

Interleaving and the standalone length header

The coded body is spread by a full-frame square block interleaver of R = floor(√N) rows, a geometry that maximizes both burst span and error spacing and grows automatically with frame length — a 250-character frame spreads bursts of roughly 600 ms while presenting the Viterbi decoder with isolated errors about 59 bits apart. Since the grid geometry depends on the payload length, the length must be known before anything can be de-interleaved. It therefore travels outside the body as a standalone 60-bit block right after the sync word: two length bytes plus a CRC-8, convolutionally coded but not interleaved. The CRC-8 rejects a corrupted-but-plausible length before it can scramble the de-interleave geometry.

Chase combining (time diversity)

The modulator’s repeat flag transmits the entire frame N times, separated by silence quantized to whole symbols so every copy lands on the receiver’s continuous symbol grid. Each copy is independently acquirable — it carries its own preamble and sync. The decoder climbs a two-rung ladder: **selection** first (decode each copy alone; first CRC pass wins), then **combining** (if every copy fails, sum the copies’ bipolar soft streams position-by-position and Viterbi-decode the sum once). Because the soft values are sign × reliability, the sum is maximal-ratio combining: a strong bit in one copy outvotes a faded bit in another, erasures contribute nothing, and two confident disagreements cancel honestly to an erasure. In steady noise two combined copies are worth about 3 dB over either alone; against bursts, each coded bit takes the best evidence any copy has. The same ladder rescues the length header.

Timing recovery and squelch

Acquisition runs in two passes: a coarse scan over 16 fractional symbol phases followed by fine phase acquisition over the preamble, then a gated early/late tracker holds the symbol grid through the capture. The receive daemon monitors the SDR continuously with energy-based squelch and hysteresis, and is hardened against USB disconnects with retry, backoff, and salvage logic so an

unattended station recovers on its own.

Implementation notes

Both ends are written in portable C with zero heap allocation — all working storage is statically sized from the maximum payload at link time — and the on-air format invariants are enforced with compile-time static assertions, so a parameter change that would break interoperability fails the build loudly. Maximum payload is 2,048 characters per frame.

Key parameters

Parameter	Value
Modulation	Binary AFSK, Bell 202 tones (1200 / 2400 Hz)
Baud rate	100 or 300 (compile-time; PALMETTO-100 / -300)
Sample rate	48,000 Hz
FEC	Convolutional K=7, rate 1/2, generators 171/133 octal
Decoder	Soft-decision Viterbi with erasure flooring
Interleaver	Full-frame square block, R = floor(sqrt(N))
Sync word	0xD3D3 (16 bits) after a 128-bit preamble
Length header	24 info bits + CRC-8, coded to 60 bits, not interleaved
Payload integrity	CRC-16 (CCITT 0x1021) over the original text
Text packing	7 bits/character (ASCII)
Max payload	2,048 characters
Time diversity	Up to 8 tracked copies; selection then chase combining

3. My Current Setup

Transmit station

A Raspberry Pi 5 drives a Yaesu FT-2900R 2-meter FM transceiver through a Digirig interface, which handles both the audio path and PTT keying. The Pi runs `fsk_mod.c`, the C modulation engine that builds the frame, applies the FEC and interleaving, and synthesizes the AFSK waveform (including on-air repeats and whole-symbol gaps for chase combining), fronted by `FSKModIc.py`, the Python GUI used to compose messages and control transmission.

Receive station

A Mac mini (M4) with a Nooelec NESDR SMArt v5 RTL-SDR dongle runs the always-on mailbox side. `fsk_rxd.c` is the capture daemon: it monitors the channel continuously with energy-based squelch, records incoming frames automatically, and survives USB disconnects with retry/backoff/salvage logic. `fsk_demod.c` is the decode engine — timing recovery, soft-decision Viterbi, de-interleaving, and chase combining — and `FSKDemodIc.py` is the Python GUI for reviewing captures and decoded messages.

Installation directory

Each station's installation directory must contain an `Archive` folder. Create it when setting up a new station — the software expects it to exist:

```
<install directory>/
    fsk_mod.c   FSKModIc.py        (transmit station)
    fsk_rxd.c   fsk_demod.c   FSKDemodIc.py  (receive station)
    Archive/
```
